day3-hive-pig

Day 3: Apache Hive and Apache Pig

Answers to the 15 Day 3 Knowledge Review questions, with short HiveQL and Pig Latin examples.

Q1. Explain schema-on-read. How does it differ from schema-on-write, and what trade-offs does it bring?

With **schema-on-read**, files are stored as they arrive and the table schema is applied only when a query reads them. Hive keeps the schema in its metastore and uses a SerDe to interpret the bytes. There is still a schema; it is just checked late.

	Schema-on-write (RDBMS)	Schema-on-read (Hive)
When schema is checked	At insert	At query time
Load speed	Slower (validate, convert)	Fast (copy files as-is)
Query speed	Faster (data already clean)	Slower (parse every read)
Bad data	Rejected at load	Shows up as NULLs or wrong values at query time
Schema change	Migration needed	New table definition over the same files

Trade-off: flexible and fast ingestion of messy, changing data, paid for with read-time parsing cost and later discovery of data quality problems.

Q2. Describe the role of the Hive metastore. What happens if it is unavailable?

The metastore is Hive's catalog. It stores databases, tables, columns and types, partitions and their locations, the storage format and SerDe, and statistics. It is usually backed by a relational database (MySQL or PostgreSQL in production, Derby only for testing). It does not store the data files.

During semantic analysis the compiler asks the metastore whether tables and columns exist and which partitions to read.

If it is down:

- New queries fail at compile time, because Hive cannot resolve table names or file locations.
- DDL (CREATE, ALTER, ADD PARTITION) fails.
- Other tools sharing it (Spark SQL, Trino) also lose their table definitions.
- The data in HDFS is safe; only access through table names is lost.

Production setups run several metastore services against a replicated database to avoid this single point of failure.

Q3. Explain the difference between managed and external tables. Why does it matter for the data lifecycle?

	Managed (internal)	External
Who owns files	Hive	Another system or team
Location	Hive warehouse directory	Given with `LOCATION`
`DROP TABLE`	Deletes metadata and data files	Deletes metadata only; files stay
Use when	Hive creates and fully controls the data	Data is shared, produced by other jobs, or must survive a drop

```
CREATE EXTERNAL TABLE raw_sales (
  sale_id BIGINT, product_id BIGINT, amount DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION '/data/landing/sales';
```

It matters because a mistaken `DROP TABLE` on a managed table destroys the data. External tables protect shared data from that.

Q4. Explain partitioning and bucketing. Give a query that benefits from each.

Partitioning stores each value of a partition column in its own directory (`dt=2024-03-15/region=North/`). A filter on that column lets Hive skip every other directory (partition pruning).
Bucketing hashes a column into a fixed number of files inside each partition. Rows with the same key always land in the same bucket file, which helps joins and sampling.

```
CREATE TABLE sales (
  sale_id BIGINT, product_id BIGINT, store_id BIGINT, amount DOUBLE
)
PARTITIONED BY (dt STRING, region STRING)
CLUSTERED BY (product_id) INTO 32 BUCKETS
STORED AS ORC;
```

- Benefits from partitioning: `SELECT SUM(amount) FROM sales WHERE dt = '2024-03-15';` reads one day, not the whole year.
- Benefits from bucketing: a join of `sales` with `products` on `product_id`, when both are bucketed on `product_id` with compatible bucket counts, can run as a bucket map join without a full shuffle. Also `TABLESAMPLE(BUCKET 1 OUT OF 32 ON product_id)` reads about 1/32 of the data.

Keep partition columns low to moderate in cardinality; too many partitions overload the metastore.

Q5. Describe row-oriented vs column-oriented storage. Give a workload where each fits.

	Row-oriented (Text, CSV, Avro)	Column-oriented (Parquet, ORC)
Layout	All fields of a row together	All values of a column together
Reads only needed columns	No	Yes (projection)
Compression	Moderate	High (similar values sit together)
Skip data with min/max stats	No	Yes (predicate pushdown)
Good for	Writing or reading whole records	Scans, filters and aggregations on a few columns

- Row-oriented fit: an ETL step or event stream (Kafka to Avro) that processes every field of each record.
- Column-oriented fit: `SELECT region, SUM(amount) FROM sales GROUP BY region` on a 200-column table; only two columns are read.

Q6. Compare Hive and Pig across abstraction, schema handling and typical user.

	Hive	Pig
Language	HiveQL, declarative and SQL-like	Pig Latin, procedural dataflow
You describe	The result you want	The steps to get it
Schema	Declared once in the metastore, shared across tools	Declared in each script with `LOAD ... AS (...)`, optional
Data model	Tables, partitions, buckets	Relations of tuples, bags, maps
Typical user	Analysts, BI users who know SQL	Data engineers writing ETL pipelines
Best for	Reporting and ad hoc analysis	Multi-step transformations, nested data

Both compile to jobs on the same cluster engine (MapReduce, Tez, or Spark depending on version and config), so the choice is mostly about authoring style and team skills.

Q7. What are the main categories of UDFs in Hive, and what does each do?

Type	Input to output	Example
UDF (scalar)	One row, one value	`mask_phone(phone)`
UDAF (aggregate)	Many rows, one value per group	A custom weighted median
UDTF (table-generating)	One row, many rows	`explode(tags)` turns an array into rows

Use built-in functions first; custom UDFs must be packaged as a jar, registered, deployed to every node and maintained, and the optimizer treats them as black boxes. Keep them deterministic.

```
ADD JAR /libs/my-udfs.jar;
CREATE TEMPORARY FUNCTION mask_phone AS 'com.example.MaskPhone';
SELECT mask_phone(phone) FROM customers;
```

Q8. Explain the role of SerDes. How does a SerDe interact with the metastore schema?

A SerDe (serializer/deserializer) converts between bytes in a file and the row objects Hive works with. The deserializer is used on read, the serializer on write.

The metastore holds two things per table: the schema (column names and types) and which SerDe to use. At read time the SerDe parses each record and maps its fields onto the declared columns. The same bytes can mean different things under a different SerDe or schema.

Example: the line `2024-03-15|North|450.00` reads as three columns with a delimited text SerDe using `|`, but fails or returns NULLs with a JSON SerDe. Common SerDes: LazySimpleSerDe (delimited text), OpenCSVSerde, JsonSerDe, RegexSerDe, plus Avro, Parquet and ORC SerDes.

Q9. Compare HiveQL and Pig Latin for a multi-stage ETL pipeline. Which is more natural, and why?

Pig Latin is usually more natural for multi-stage ETL. Each step is a named relation, so cleaning, branching (`SPLIT`), joining and reshaping read top to bottom like a pipeline, and nested types (bags, tuples, maps) need no flattening first. In HiveQL the same pipeline becomes nested subqueries or several intermediate tables.

Same aggregation, revenue per category for one day:

```
-- HiveQL
SELECT p.category, SUM(s.amount) AS revenue
FROM sales s JOIN products p ON s.product_id = p.product_id
WHERE s.dt = '2024-03-15'
GROUP BY p.category;
```

```
-- Pig Latin
sales    = LOAD '/data/sales' USING PigStorage(',')
           AS (sale_id:long, product_id:long, amount:double, dt:chararray);
products = LOAD '/data/products' USING PigStorage(',')
           AS (product_id:long, name:chararray, category:chararray);
day      = FILTER sales BY dt == '2024-03-15';
joined   = JOIN day BY product_id, products BY product_id;
by_cat   = GROUP joined BY products::category;
revenue  = FOREACH by_cat GENERATE group AS category, SUM(joined.day::amount) AS revenue;
STORE revenue INTO '/out/revenue_by_category' USING PigStorage(',');
```

For a single report the HiveQL is shorter and readable by any SQL user; for a long chain of transformations the Pig script is easier to follow and debug step by step.

Q10. Why is Hive not suited to interactive, low-latency workloads? Which design decisions cause this?

- Every query is compiled into distributed jobs; start-up and scheduling alone can take seconds.
- It was built for full scans of large data, optimising throughput, not response time.
- Schema-on-read means parsing and deserialising data at query time.
- There are no row-level indexes for point lookups like an OLTP database.
- Joins and aggregations shuffle data across the network, with intermediate results written to disk (especially on MapReduce).
- Data lives in HDFS files, not in a memory-resident store.

Hive on Tez with LLAP caching and columnar formats can bring some queries down to seconds, but sub-second, high-concurrency workloads belong on tools such as Trino/Presto, Impala, Druid or a relational database.

Q11. A Hive table has 500 partitions, and a query filters on a partition column whose values are not in the file paths. Diagnose and fix.

Likely problem: the partition metadata and the directory layout do not match. Partition columns are not stored inside the data files; Hive gets their values from the metastore and the `col=value` directory names. If the files were written straight into the table folder (not into `dt=.../` subfolders), or partitions were never registered, Hive either finds nothing or scans everything, and pruning does not work.
Fix:

- Lay the files out as `/table/dt=2024-03-15/...` so the directory name carries the value.
- Register partitions: `MSCK REPAIR TABLE sales;` or `ALTER TABLE sales ADD PARTITION (dt='2024-03-15') LOCATION '...';`
- Or reload with a dynamic partition insert so Hive creates the layout:

```
SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE sales PARTITION (dt)
SELECT sale_id, product_id, store_id, amount, dt FROM raw_sales;
```

- Write the filter directly on the column (`WHERE dt = '2024-03-15'`), not wrapped in a function, so pruning can apply. Check with `EXPLAIN` and `SHOW PARTITIONS sales`.

Q12. A Pig script produces a very large intermediate relation between two joins. How do you tell whether shuffle volume or skew is the cause, and what do you try first?

Diagnose:

- Run `EXPLAIN` to see the physical plan and how many jobs and shuffles there are.
- Use `DESCRIBE` and `ILLUSTRATE` on a sample to see the shape and size of each relation.
- Check job counters: map output records and bytes show shuffle volume.
- Compare reduce task durations. If one or two reducers run far longer than the rest, it is skew. If all are slow and evenly loaded, it is volume.

First fixes:

- Reduce volume early: `FILTER` before the joins, and `FOREACH ... GENERATE` only the needed fields.
- If one side is small, use `JOIN ... USING 'replicated'` to avoid the shuffle.
- If it is skew on a hot key, use `JOIN ... USING 'skewed'`.
- Check for a key that explodes the join (many-to-many or NULL-heavy key) and fix the join key.

Q13. Queries on an inherited Hive table return strange NULLs in some columns. Where do you look first, and what does it suggest?

Look at the table definition first: `SHOW CREATE TABLE t;` or `DESCRIBE FORMATTED t;`, then compare it with a few raw lines from the files (`hdfs dfs -cat /path/part-00000 | head`).

It suggests a schema or SerDe mismatch, because under schema-on-read Hive returns NULL when a field cannot be parsed into the declared type. Typical causes:

- Wrong field delimiter, so fields shift or merge.
- Column order in the DDL differs from the file.
- Type mismatch (text like `"1,200"` declared as INT, odd timestamp formats).
- Schema evolved: older files lack new columns.
- Wrong SerDe (CSV read with LazySimpleSerDe, quoted fields not handled).
- Header row not skipped (`skip.header.line.count`).

Q14. Hive or Pig for a pipeline that ingests heterogeneous JSON, flattens nested fields and writes Parquet? Why?

Recommend Pig for the transformation itself. JSON with nested and varying structure maps naturally to Pig's tuples, bags and maps, and a step-by-step script (load, clean, `FLATTEN`, split good and bad records, store) is easier to write and debug than nested HiveQL with `LATERAL VIEW explode`.

Hive is still a fair choice if the team only knows SQL: a JSON SerDe plus `LATERAL VIEW explode(...)` can flatten the data, and `INSERT ... STORED AS PARQUET` writes the output. Either way, register the final Parquet as an external Hive table so analysts can query it. In many teams today the same job would be done in Spark.

Q15. An analyst asks why Hive queries take minutes, not seconds. Explain, and say what can and cannot be done.

Why: each query compiles into distributed jobs with start-up cost, scans large files from HDFS, parses data at read time, and shuffles data between stages. Hive was designed for throughput on large batch scans, not for fast single answers.

What can be done:

- Use Tez (or Spark) instead of MapReduce, and LLAP caching where available.
- Store data as ORC or Parquet with compression.
- Partition by common filters (date) and filter on partition columns.
- Select only needed columns; avoid `SELECT *`.
- Keep table statistics up to date (`ANALYZE TABLE ... COMPUTE STATISTICS`) and enable cost-based optimisation.
- Use map joins for small tables, bucketing for large joins.
- Pre-aggregate into small summary tables for dashboards.

What cannot reasonably be done: make Hive an OLTP or sub-second, high-concurrency engine. For that, serve results from Trino/Presto, Impala, Druid, or a relational database fed by Hive.

Quick recall

- Hive is a SQL query layer over files; the engine underneath is MapReduce, Tez or Spark.
- Schema-on-read: schema exists, checked when reading, not loading.
- Metastore holds metadata only; backed by MySQL or PostgreSQL in production.
- Dropping a managed table deletes data; dropping an external table keeps it.
- Partition = directory per value, enables pruning. Bucket = hash into fixed files, helps joins and sampling.
- Partition columns are not stored inside the data files.
- Parquet and ORC are columnar: read fewer columns, compress better, skip blocks.

- SerDe converts bytes to rows and back; wrong SerDe or delimiter gives NULLs.
- UDF one-to-one, UDAF many-to-one, UDTF one-to-many (e.g. `explode`).
- Pig Latin is procedural; a bag is an unordered collection of tuples.
- Pig join options: `replicated` for small tables, `skewed` for hot keys.
- Hive suits batch analytics; it is not built for sub-second queries.